

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Programación 1

Trabajo Práctico Integrador

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Alumnos

Antonella Anahi Lopez - M26 C1-14

Correo: antotolo15@gmail.com

Santiago Jair Vidal - M26 C1-22

Correo: vidal.santiago.jair@gmail.com

xx de junio de 2026

Tabla de contenido

Introducción

Objetivo

Marco Teórico

 Listas

 Diccionarios

 Funciones

 Condicionales

 Ordenamientos

 Estadísticas Básicas

 Archivos CSV

Decisiones Técnicas y Arquitectura

 Desarrollo del Proyecto (diagrama)

 Capturas de Ejecución

Dificultades Encontradas

Conclusiones

Bibliografía / Webgrafía

Links del Proyecto

Introducción

El presente trabajo práctico fue desarrollado con el objetivo de aplicar conceptos fundamentales de programación utilizando el lenguaje Python.

El sistema implementado permite gestionar información de distintos países mediante el uso de archivos de texto, listas, diccionarios y funciones. Además, el proyecto busca fortalecer conocimientos relacionados con la organización de datos, reutilización de código y persistencia de información.

El desarrollo fue realizado utilizando Visual Studio Code y posteriormente almacenado en GitHub para mantener un control de versiones adecuado.

Objetivo

General

Desarrollar un programa en Python capaz de leer, almacenar y procesar información de países utilizando estructuras de datos y archivos de texto.

Específicos

Comprender el funcionamiento de listas y diccionarios.

Aplicar funciones para modularizar el código.

Leer y escribir información en archivos de texto.

Organizar los datos de manera eficiente.

Implementar búsquedas y visualización de información.

Marco Teórico

Listas

Las listas son estructuras de datos que permiten almacenar múltiples elementos dentro de una misma variable.

En Python, las listas son dinámicas y pueden modificarse agregando o eliminando elementos.

Ejemplo:

```
países = ["Argentina", "Brasil", "Japón"]
```

Las listas fueron utilizadas para almacenar los registros cargados desde el archivo de texto.

Diccionario

Los diccionarios permiten almacenar datos mediante pares clave-valor, facilitando el acceso organizado a la información.

Ejemplo:

```
país = {  
  
    "nombre": "Argentina",  
  
    "poblacion": 45376763,  
  
    "superficie": 2780400,
```

```
“continente”: "América"
```

```
}
```

Cada país fue almacenado dentro del programa como un diccionario independiente.

Funciones

Las funciones son bloques de código reutilizables que permiten dividir un programa en tareas específicas.

Ejemplo:

```
def mostrar_pais(nombre):
```

```
    print(nombre)
```

En el proyecto se utilizaron funciones para:

- Leer archivos
- Mostrar datos
- Buscar países
- Agregar información

Condicionales

Los condicionales son estructuras de control que permiten evaluar si una o más condiciones son verdaderas o falsas, derivando el flujo del programa según el resultado.

TRABAJO PRÁCTICO INTEGRADOR - PROGRAMACIÓN 1

En Python se utilizan las palabras clave if, elif y else.

Ejemplo:

```
if pais_actual["continente"] == "América":
```

```
    contador_america += 1
```

```
elif pais_actual["continente"] == "Europa":
```

```
    contador_europa += 1
```

```
else:
```

```
    contador_otros += 1
```

En el proyecto, los condicionales se utilizarán para comparar los registros entre sí permitiendo identificar cuáles son los países con mayor o menor población y clasificar de forma correcta los totales de cada continente, entre otras cosas.

Ordenamientos

Los ordenamientos son algoritmos que organizan los elementos de una colección bajo un criterio específico (alfabético o numérico) de forma ascendente (menor a mayor) o descendente (mayor a menor)

Ejemplo:

```
for i in range(len(paises)):
```

```
    for j in range(len(paises) - i - 1):
```

```
if paises[j]['poblacion'] < paises[j+1]['poblacion']:
```

```
    temporal = paises[j]
```

```
    paises[j] = paises[j+1]
```

```
    paises[j+1] = temporal
```

En el programa, esta estructura lógica basada en el método de bubble short será la encargada de resolver el sistema de ordenamiento multifunción.

Estadísticas Básicas

Consisten en la aplicación de operaciones matemáticas y lógicas sobre un conjunto de datos para extraer métricas descriptivas globales del conjunto.

Ejemplo:

```
cantida = len(paises)
```

```
suma = sum(pais[clave] for pais in paises)
```

```
prom = suma / cantida
```

En el programa, estas operaciones se agruparán dentro de una función específica de reportes que será la encargada de resolver el sistema de análisis estadístico.

Archivos CSV

TRABAJO PRÁCTICO INTEGRADOR - PROGRAMACIÓN 1

Un archivo CSV (valores separados por comas) es un formato de texto plano diseñado para almacenar datos en forma de tabla. Es ideal para lograr persistencia de la información, permitiendo que los datos no se borran al cerrar el programa.

Ejemplo de estructura interna del archivo(países.csv):

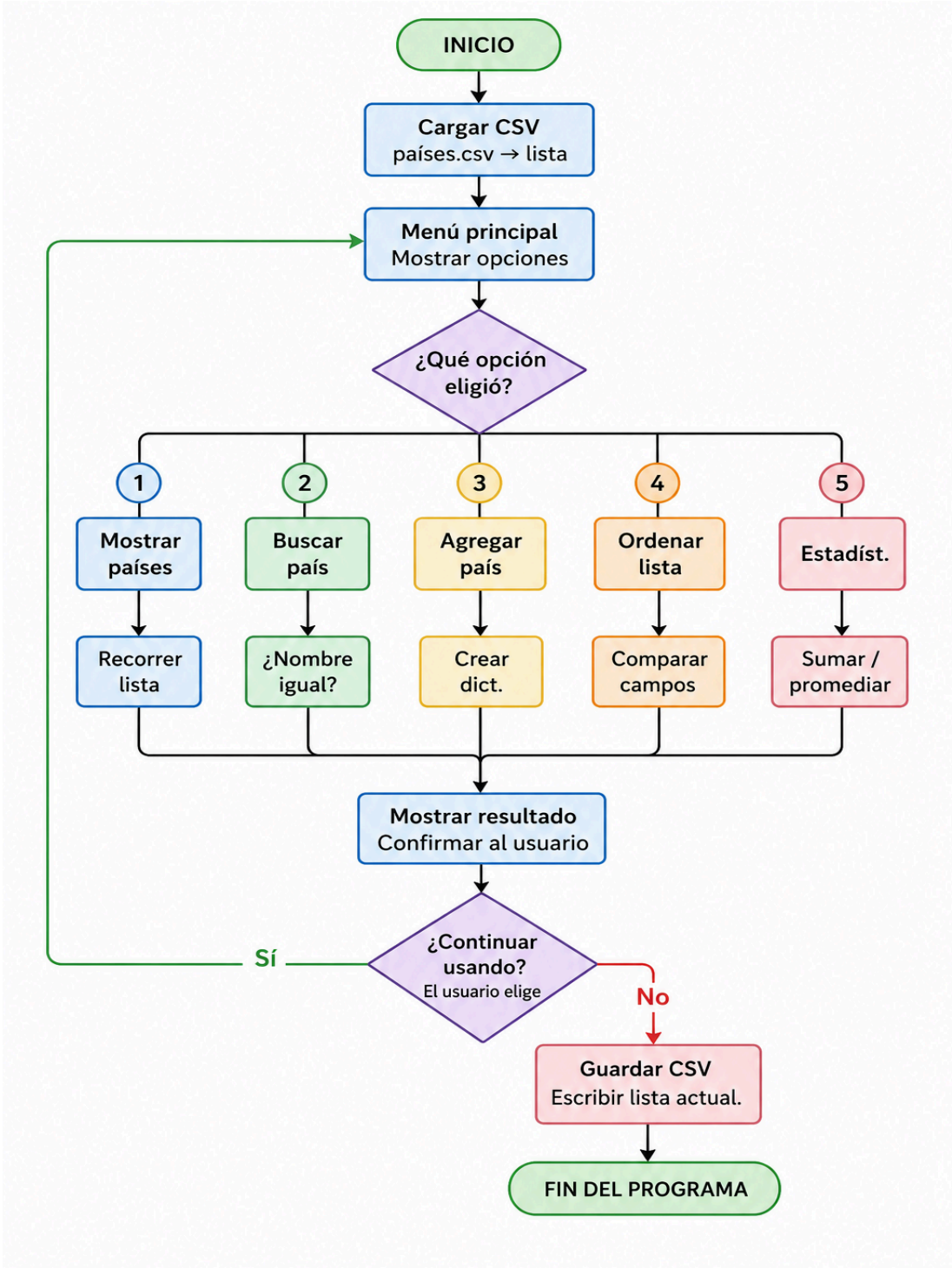
nombre,población,superficie,continente

Argentina,45376763,2780400,América

Japón,125800000,377975,Asia

En el proyecto este archivo actuará como base de datos. Al iniciar el programa leerá el archivo CSV y convertirá cada fila en un diccionario dentro de una lista. Al finalizar el programa tomará la lista actualizada y la escribirá nuevamente en el archivo csv para salvar los cambios.

Desarrollo del Proyecto



Capturas de Ejecución

Ejemplo de la ejecución de la opción 1 (Agregar países):

```
=====
1. Agregar País.
2. Actualizar Datos.
3. Buscar País.
4. Filtrar Países.
5. Ordenar Países.
6. Estadísticas.
7. Salir.
=====
Ingrese la opcion que quiere ejecutar: 1
Ingrese el nombre del pais: Japón
Ingrese la poblacion del pais: 125800000
Ingrese la superficie del pais (en km²): 377975
Ingrese el continente del pais: Asia

El pais "Japón" fue agregado exitosamente.
-----
Nombre      : Japón
Poblacion   : 125,800,000 habitantes
Superficie  : 377,975 km²
Continente  : Asia
-----
=====
```

Ejemplo de la ejecución de la opción 3 (Busqueda):

```
=====
1. Agregar País.
2. Actualizar Datos.
3. Buscar País.
4. Filtrar Países.
5. Ordenar Países.
6. Estadísticas.
7. Salir.
=====
Ingrese la opcion que quiere ejecutar: 3
Ingrese el nombre del pais a buscar: japon

Se encontraron 1 resultado/s:
-----
Nombre      : Japon
Poblacion   : 125,800,000 habitantes
Superficie  : 377,975 km²
Continente  : Asia
-----
=====
```

TRABAJO PRÁCTICO INTEGRADOR - PROGRAMACIÓN 1

Ejemplo de la ejecución de la opción 4 (Filtrado) apartado 1 (Filtrar por continente):

```
=====
1. Agregar Pais.
2. Actualizar Datos.
3. Buscar Pais.
4. Filtrar Países.
5. Ordenar Países.
6. Estadísticas.
7. Salir.
=====
Ingrese la opcion que quiere ejecutar: 4
=====
1. Filtrar por continente.
2. Filtrar por rango de poblacion.
3. Filtrar por rango de superficie.
=====
Ingrese la opcion que quiere ejecutar: 1
Ingrese el nombre del continente con el que quiere filtrar: america
Pais:  Argentina | Poblacion:  45376763 | Superficie:  2780400 | Continente:  América
Pais:   Brasil   | Poblacion:  213993437 | Superficie:  8515767 | Continente:  América
Pais:   México   | Poblacion:  130262216 | Superficie:  1964375 | Continente:  América
Pais: Estados Unidos | Poblacion:  331893745 | Superficie:  9833517 | Continente:  América
=====
```

Ejemplo de la ejecución de la opción 6 (Estadísticas) apartado 2 (Promedios):

```
=====
1. Agregar Pais.
2. Actualizar Datos.
3. Buscar Pais.
4. Filtrar Países.
5. Ordenar Países.
6. Estadísticas.
7. Salir.
=====
Ingrese la opcion que quiere ejecutar: 6
=====
1. Pais con mayor y menor poblacion.
2. Promedios(Poblacion y superficie).
3. Cantidad de paises por continente.
=====
Ingrese la opcion que quiere ejecutar: 2
=====
PROMEDIOS:
=====
Promedio de Poblacion: 130,110,414.44 hab.
Promedio de Superficie: 2,807,190.67 km²
=====
```

Dificultades

Las principales dificultades que se encontraron fueron la forma de trabajar juntos ya que al tener distintos horarios se dificultó la comunicación y también la distinta forma de trabajar a la hora de programar.

Conclusión

En conclusión fue un desafío nuevo tener que trabajar en conjunto teniendo diferentes horarios y formas de trabajar, aun así de todas formas se pudo resolver los puntos del trabajo.

Bibliografía / Webgrafía

Listas y Diccionarios (Estructura de datos):

[Python Software Foundation](#). Data Structures — Python Documentation.

Sección oficial sobre el funcionamiento, métodos y mutabilidad de estructuras de datos lineales y asociativas en Python.

Funciones:

[Python Software Foundation](#). Defining Functions — Python Documentation.

Guía oficial sobre la sintaxis de declaración (def), el uso de argumentos y el retorno de valores en bloques modulares de código.

Estructuras de Control Condicionales:

[Python Software Foundation](#). More Control Flow Tools: if Statements.

Documentación oficial sobre la lógica de ramificación del flujo de un programa mediante las estructuras if, elif y else.

Ordenamiento:

[Abel Garrido / Python in Plain English](#). Python Sorting Algorithms: Bubble Sort.

Análisis conceptual y técnico sobre la lógica de comparación lineal e intercambio de posiciones en el algoritmo de ordenamiento de burbuja.

Archivo CSV:

[Python Software Foundation](#) (Español). csv — CSV File Reading and Writing.

Documentación oficial en español del módulo nativo para lectura y escritura de archivos planos estructurados por filas y columnas.

Links del Proyecto

Link del repositorio github: [Repositorio--Lopez-Antonella--Vidal-Santiago](#).

Link del Video Explicativo:

Mi compañera decidió hacer el video por su cuenta así que este link lleva al video explicativo explicando el funcionamiento completo por Santiago Vidal.

[Video-Explicativo-del-TPI-Programacion1](#).